Array

Lecture 12

Based on Slides of Dr. Norazah Yusof

Introducing Arrays

- **Array is a data structure that represents a collection of the same types of data.**
- **In Java, array is an object that can store a group of values, all of the same type.**

Declare and create an array

- **Declare and create array is similar to create and create any other type of object.**
- **Example question:**

  Declare and create 10-element double array referenced by the variable, myList.

Declare and create an array in two steps

Step 1: declare an array reference

```
datatype[] arrayRefVar;
```

or

```
datatype arrayRefVar[];  // This style is allowed,
                         // but not preferred
```

Step 2: create an array

```
arrayRefVar = new datatype[arraySize];
```

Example

```
double[] myList;
myList = new double[10];
```

Declare and create an array in one step

- **Declare and create the array**

```
datatype[] arrayRefVar = new datatype[arraySize];
```

or

```
datatype arrayRefVar[] = new datatype[arraySize];
```

- **Example**

```
double[] myList = new double[10];
```

or

```
double myList[] = new double[10];
```

The Length of an Array

Once an array is created, its size is fixed. It cannot be changed. You can find its size using

```
arrayRefVar.length
```

For example,

```
myList.length
```

returns 10

Default Values

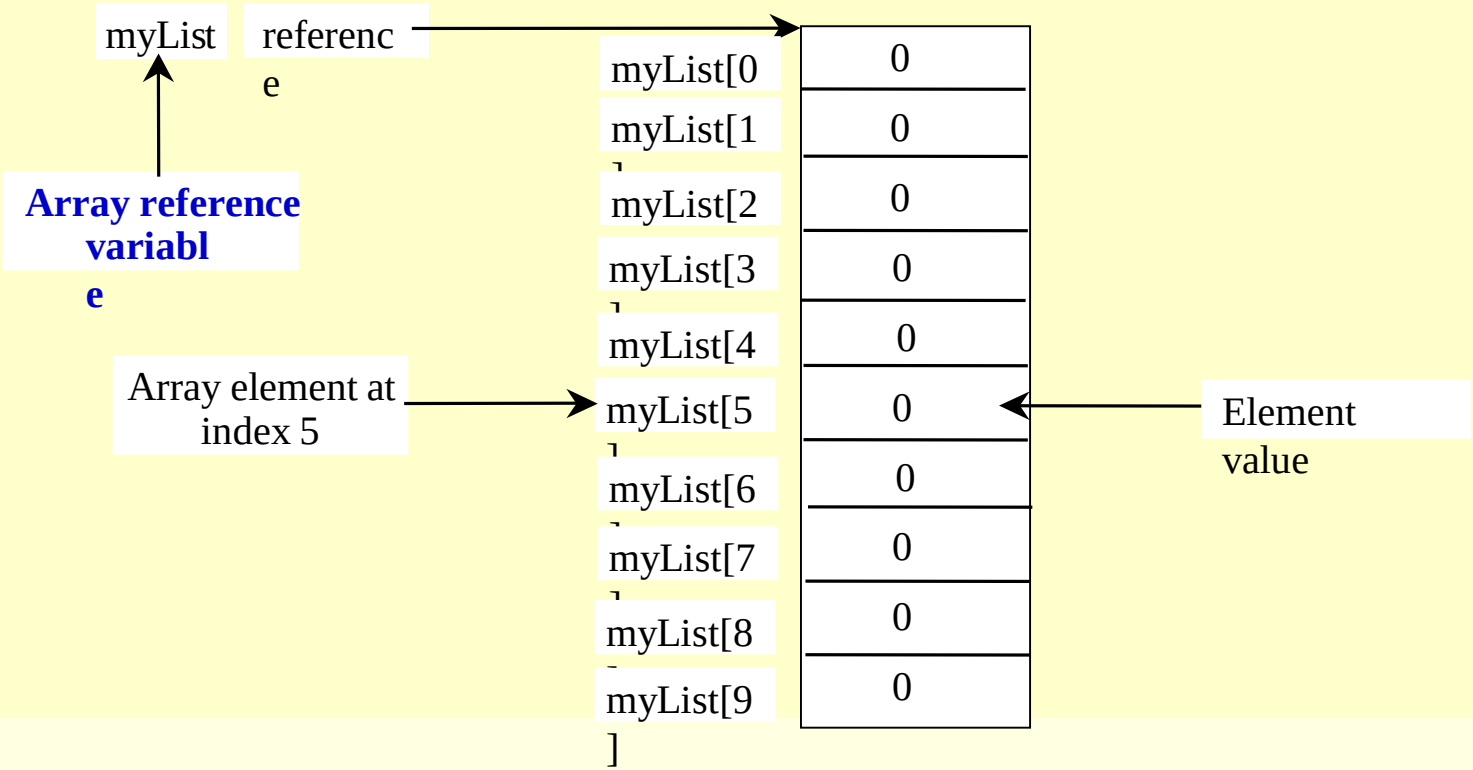
When an array is created, its elements are assigned the default value of

- `0` **for the numeric primitive data types,**
- `'\u0000'` **(unicode value) for <u>char</u> types, and**
- `false` **for <u>boolean</u> types.**

Arrays

- **Subscript**
  - **Integer contained within square brackets**
  - **Indicates one of array's variables or elements**

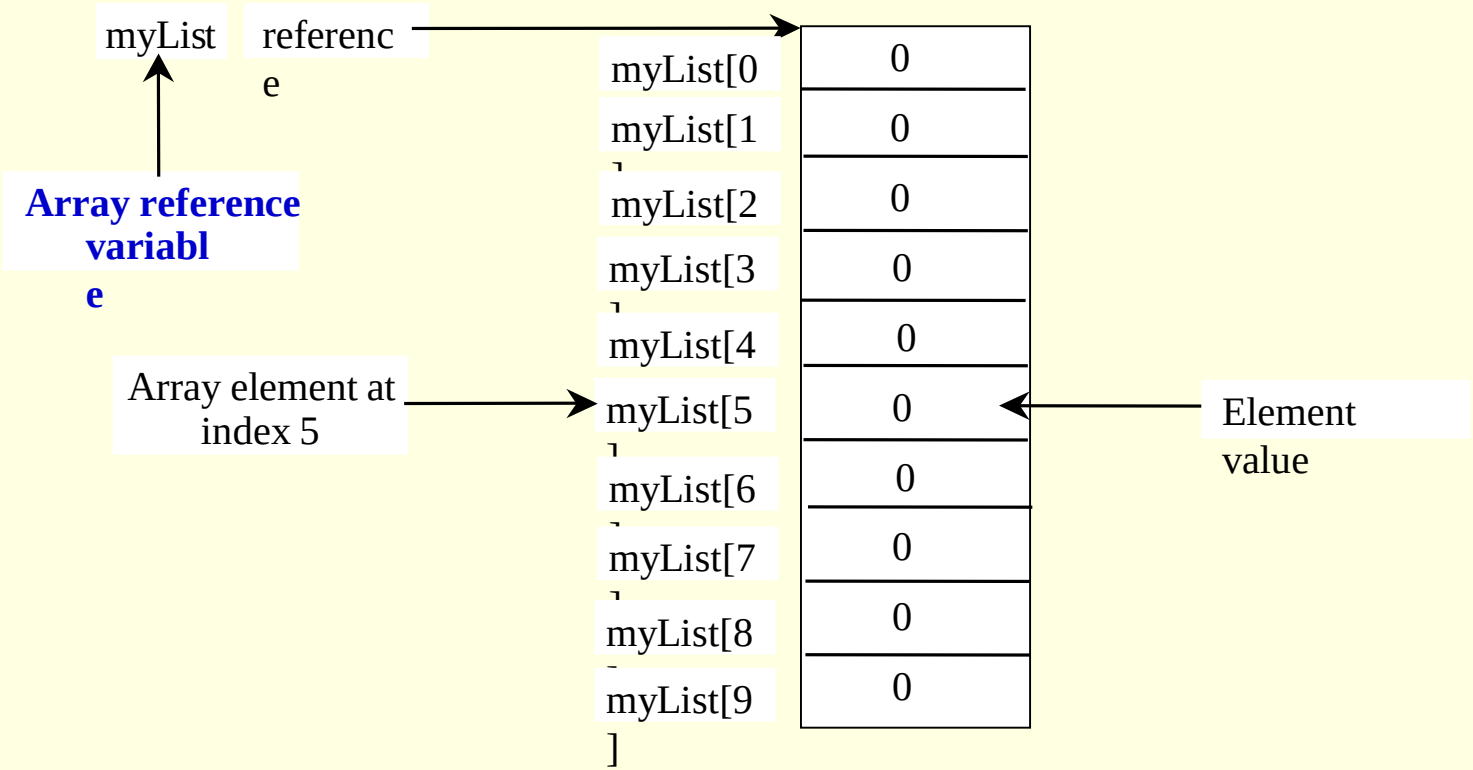
```
double[] myList = new double[10];
```



Arrays

- **Array's elements numbered beginning with zero**
  Can legally use any subscript from 0 through `myList.length-1`

```
double[] myList = new double[10];
```



Accessing Array Elements

- **The array elements are accessed through the index, known as indexed variable**

```
arrayRefVar[index];
```

- **An indexed variable can be used in the same way as a regular variable.**
- **For example, the value of the array element may be assigned as follows:**

```
myList[0] =  5.6;
myList[1] =  myList[0] - 1.1;
Scanner inp = new Scanner(System.in);
myList[2] = inp.nextDouble();
```

Accessing Array Elements

- **The array elements can be output using the println method.**

```
System.out.println (myList[0]);
System.out.println (myList[1]);
System.out.println (myList[2]);
```

Write & Trace the output

```
class ArrayDemo {
    public static void main(String[] args) {
        int[] anArray; // declares an array of integers
        anArray = new int[10]; // allocates memory for 10 integers
        anArray[0] = 100; // initialize first element
        anArray[1] = 200; // initialize second element
        anArray[2] = 300; // etc.
        anArray[3] = 400;
        anArray[4] = 500;
        anArray[5] = 600;
        anArray[6] = 700;
        anArray[7] = 800;
        anArray[8] = 900;
        anArray[9] = 1000;
```

```
System.out.println("Element at index 0: " + anArray[0]);
System.out.println("Element at index 1: " + anArray[1]);
System.out.println("Element at index 2: " + anArray[2]);
System.out.println("Element at index 3: " + anArray[3]);
System.out.println("Element at index 4: " + anArray[4]);
System.out.println("Element at index 5: " + anArray[5]);
System.out.println("Element at index 6: " + anArray[6]);
System.out.println("Element at index 7: " + anArray[7]);
System.out.println("Element at index 8: " + anArray[8]);
System.out.println("Element at index 9: " + anArray[9]);
    }
}
```

- **The output from this program is:**

```
Element at index 0: 100
Element at index 1: 200
Element at index 2: 300
Element at index 3: 400
Element at index 4: 500
Element at index 5: 600
Element at index 6: 700
Element at index 7: 800
Element at index 8: 900
Element at index 9: 1000
```

Using loop to cycle through the entire array

- **Perform loops that vary loop control variable**
  - **Start at 0**
  - **End at one less than size of array**
- **Using constant that is equal to size of array**

```
final int SIZE = 10;
 for(int i = 0; i < SIZE; i++)
  myList[i] += 3;
```

- **Or using the `length` field, that contains the number of elements in array**

```
for(int i = 0; i < myList.length; i++)
  myList[i] += 3;
```

Declaring, creating, initializing Using the Shorthand Notation

- **Declare and create the array**

```
datatype[] arrayRefVar =
        {arrylist1, arrylist2,…};
```

- **Example**

```
double[] myList = {5.6, 4.5, 3.3,
13.2, 4, 34.33, 34, 45.45, 99.993,
11.3};
```

Declaring, creating, initializing Using the Shorthand Notation

```
double[] myList = {5.6, 4.5, 3.3, 13.2, 4, 34.33,
      34, 45.45, 99.993, 11.3};
```

This shorthand notation is equivalent to the following statements:

```
double[] myList = new double[10];
myList[0] = 5.6;  myList[1] = 4.5; myList[2] = 3.3;
myList[3] = 13.2; myList[4] = 4;   myList[5] = 34.33;
myList[6] = 34;   myList[7] = 45.45;
myList[8] = 99.993; myList[9] = 11.3;
```

CAUTION!!

Using the shorthand notation, you have to declare, create, and initialize the array all in one statement.

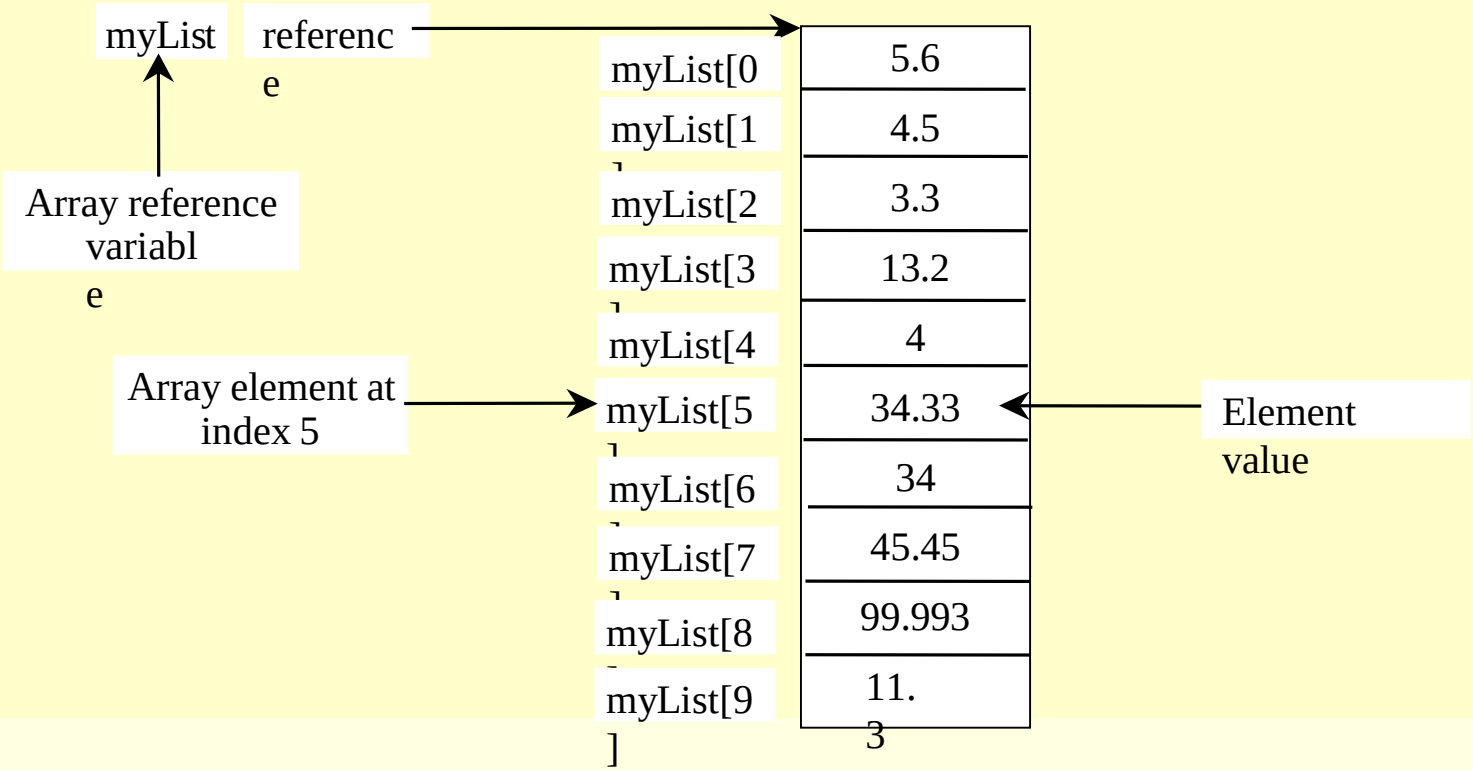
Splitting it would cause a syntax error. For example, the following is wrong:

```
double[] myList;
myList = {1.9, 2.9, 3.4, 3.5};
```

Arrays

- **Array's elements numbered beginning with zero**
  Can legally use any subscript from 0 through 9

```
double[] myList = new double[10];
```



Trace Program with Arrays

- Given the following program, what will be the output?

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

Trace Program with Arrays

Declare array variable values, create an array, and assign its reference to values

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

i becomes 1

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

i (=1) is less than 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Trace Program with Arrays

After this line is executed, value[1] is 1

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

After i++, i becomes 2

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

i (= 2) is less than 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = values[i] + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0

Trace Program with Arrays

After this line is executed,
values[2] is 3 (2 + 1)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

After this, i becomes 3.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

i (=3) is still less than 5.

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0

Trace Program with Arrays

After this line, values[3] becomes 6 (3 + 3)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

After this, i becomes 4

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

i (=4) is still less than 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0

Trace Program with Arrays

After this, values[4] becomes 10 (4 + 6)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

After i++, i becomes 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

i (=5) < 5 is false. Exit the loop

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

Trace Program with Arrays

After this line, values[0] is 11 (1 + 10)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

0	11
1	1
2	3
3	6
4	10

Enhanced for Loop

JDK 1.5 and above introduced a new for loop that enables you to traverse the complete array sequentially without using an index variable. For example, the following code displays all elements in the array `scoreArray`:

```
for(int val : scoreArray)
      System.out.println(val);
```

In general, the syntax is

```
for (elementType value: arrayRefVar) {
    // Process the value
  }
```

You still have to use an index variable if you wish to traverse the array in a different order or change the elements in the array.

Array Processing

1. (Printing arrays)
2. (Summing all elements)
3. (Finding the largest element)
4. (Finding the smallest index of the largest element)

Example: Print an Array

- **Print an array of int and an array of double:**

```
public class PrintArrays  {
   static final int ARRSIZE = 10;        // The array's size

   static int intArr[] = new int[ARRSIZE];     // Create the int a
   static double realArr[] = { 1.1, 2.2, 3.3, 4.4,
         5.5, 6.6, 7.7, 8.8, 9.9, 10.10 };     // And a double array

   public static void main(String args[]) {
      System.out.println("Ints \t Reals");
      for (int k = 0; k < intArr.length; k++)
          System.out.println( intArr[k] + " \t " + realArr[k]);
    } // main()
} // PrintArrays
```

These must be static ...

... in order to refer to them in static main()

Uninitialized int array has default values of 0.

Program Output

Ints	Reals
0	1.1
0	2.2
0	3.3
0	4.4
0	5.5
0	6.6
0	7.7
0	8.8
0	9.9
0	10.1

Example: Store the First 100 Squares

```
public class Squares {
    static final int ARRSIZE = 100;            // The array's size
    static int intArr[] = new int[ARRSIZE];    // Create an int array

    public static void main(String args[]) {
        for (int k = 0; k < intArr.length; k++)  // Initialize the
array
            intArr[k] = (k+1) * (k+1);

        System.out.print("The first 100 squares are"); // Print a
heading
        for (int k = 0; k < intArr.length; k++)
        {       // Print the array
            if (k % 10 == 0)
                System.out.println(" ");      // 10 elements per row
            System.out.print( intArr[k] + " ");
        } // for
    } // main()
} // Squares
```

Program Output

```
The first 100 squares are
1 4 9 16 25 36 49 64 81 100
121 144 169 196 225 256 289 324 361 400
441 484 529 576 625 676 729 784 841 900
961 1024 1089 1156 1225 1296 1369 1444 1521 1600
1681 1764 1849 1936 2025 2116 2209 2304 2401 2500
2601 2704 2809 2916 3025 3136 3249 3364 3481 3600
3721 3844 3969 4096 4225 4356 4489 4624 4761 4900
5041 5184 5329 5476 5625 5776 5929 6084 6241 6400
6561 6724 6889 7056 7225 7396 7569 7744 7921 8100
8281 8464 8649 8836 9025 9216 9409 9604 9801
```

Exercise 1 (Question 1)

Modify program ArrayDemo and save as ArrayDemo1

- **Initialize the elements in one initialization statement**

```
class ArrayDemo {
    public static void main(String[] args) {
        int[] anArray; // declares an array of integers
        anArray = new int[10]; // allocates memory for 10 integers
        anArray[0] = 100; // initialize first element
        anArray[1] = 200; // initialize second element
        anArray[2] = 300; // etc.
        anArray[3] = 400;
        anArray[4] = 500;
        anArray[5] = 600;
        anArray[6] = 700;
        anArray[7] = 800;
        anArray[8] = 900;
        anArray[9] = 1000;
```

```
double[] myList = {5.6, 4.5, 3.3, 13.2, 4, 34.33,
        34, 45.45, 99.993, 11.3};
```

Exercise 1 (Question 1) cont.

1. **Use traditional for loop to print all the values**

```
System.out.println("Element at index 0: " + anArray[0]);
System.out.println("Element at index 1: " + anArray[1]);
System.out.println("Element at index 2: " + anArray[2]);
System.out.println("Element at index 3: " + anArray[3]);
System.out.println("Element at index 4: " + anArray[4]);
System.out.println("Element at index 5: " + anArray[5]);
System.out.println("Element at index 6: " + anArray[6]);
System.out.println("Element at index 7: " + anArray[7]);
System.out.println("Element at index 8: " + anArray[8]);
System.out.println("Element at index 9: " + anArray[9]);
}
}
```

```
class ArrayDemo1 {
    public static void main(String[] args) {
        int anArray[] = {100, 200, 300, 400,
          500, 600, 700, 800, 900, 1000};

        for (int i = 0; i < anArray.length; i++) {
            System.out.println("Element at
          index" + i + ": " + anArray[i]);
        }
    }
}
```

Exercise 1 (Question 2) cont.

Modify program ArrayDemo1 and save as ArrayDemo2

1. **Use enhanced for loop / for-each loop to print all the values**

```
class ArrayDemo2 {
    public static void main(String[] args) {
        int anArray[] = {100, 200, 300, 400,
            500, 600, 700, 800, 900, 1000};
        for (int i: anArray) {
            System.out.println(i);
        }
    }
}
```

- **The output from this program is:**

```
Element at index 0: 100
Element at index 1: 200
Element at index 2: 300
Element at index 3: 400
Element at index 4: 500
Element at index 5: 600
Element at index 6: 700
Element at index 7: 800
Element at index 8: 900
Element at index 9: 1000
```

Passing Array to Methods

Lecture 13

Based on Slides of Dr. Norazah Yusof

Passing Array Elements to a Method

- **When a single element of an array is passed to a method it is handled like any other variable.**
- **See example: PassElements.java**

Example 1

```
1. public class PassElements {
2.   public static void main (String[] args) {
3.     int[] numbers = {5, 10, 15, 20, 25, 30, 35, 40};
4.     for (int i=0; i<numbers.length; i++) {
5.       showValue(numbers[i]);
6.     }
7.     System.out.println();
8.   }
9.
10.  public static void showValue(int num) {
11.    System.out.print((num+2) + " ");
12.  }
13.}
```

Passing Arrays as Arguments to Methods

- **More often you will want to write methods to process array data by passing the entire array, not just one element at a time.**
- **An array can be passed as an argument to a method.**
- **To pass an array, you pass the value in the variable that references the array.**
- **See example: PassArray.java**

Example 2

```
1. import java.util.*;

2. public class PassArray {
3.   public static void main (String[] args) {
4.     final int SIZE = 4;
5.     int[] numbers = new int[SIZE];
6.     getValues(numbers);
7.     System.out.println("Here are the numbers.");
8.     showArray(numbers);
9.     System.out.println();
10.   }
```

Example 2

```
12. public static void getValues(int[] arr) {
13.     Scanner kb= new Scanner(System.in);
14.     System.out.println("Enter a series of " + arr.length +
15.         " numbers.");
16.     for (int i=0; i<arr.length; i++) {
17.         System.out.print("Enter number " + (i+1) + ":");
18.         arr[i] = kb.nextInt();
19.     }
20.   }
21. public static void showArray(int[] arr) {
22.     for (int i=0; i<arr.length; i++) {
23.         System.out.print(arr[i] + " ");
24.     }
25.   }
26.}
```

Pass By Value

- **Java uses *pass by value* to pass parameters to a method.**
- **There are important differences between passing a value of variables of primitive data types and passing arrays.**

Pass By Value

- **For a parameter of a primitive type value, the actual value is passed. Changing the value of the local parameter inside the method does not affect the value of the variable outside the method.**
- **For a parameter of an array type, the value of the parameter contains a reference to an array; this reference is passed to the method. Any changes to the array that occur inside the method body will affect the original array that was passed as the argument.**

Lab 5 – Exercise 1 – Question 1(i)

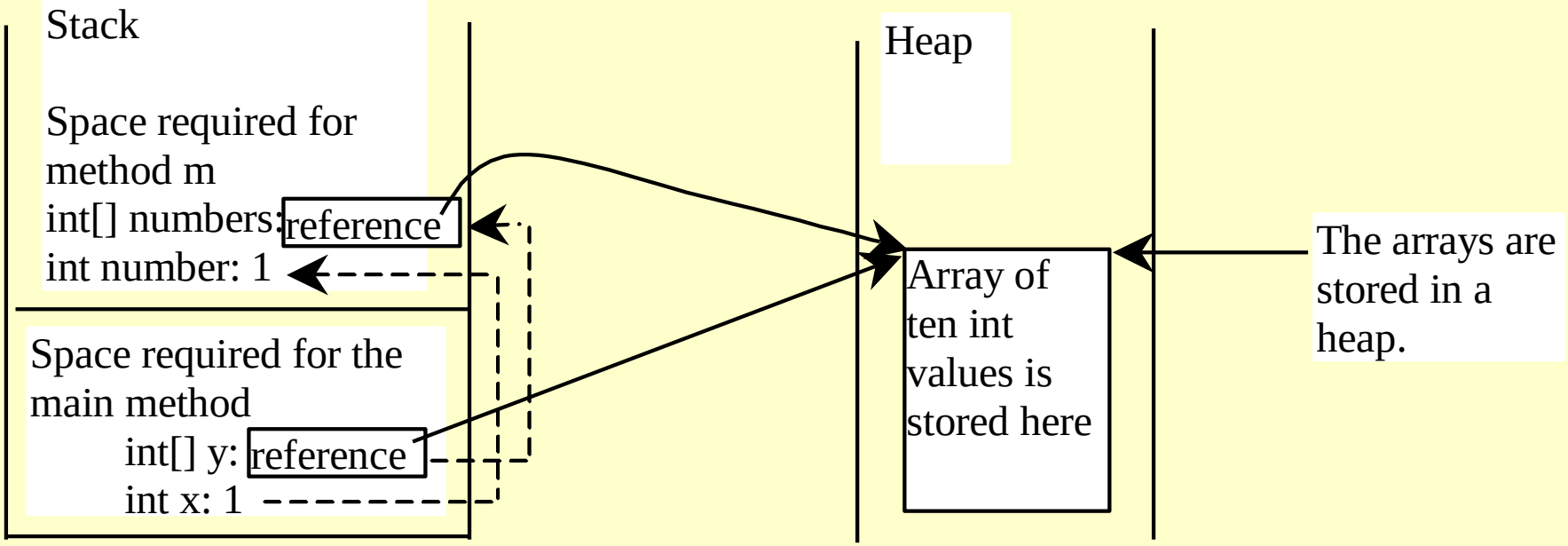
```
public class Lab7Ex1_1i {
  public static void main(String[] args) {
    int x = 1; // x represents an int value
    int[] y = new int[10]; // y represents an array of int values

    m(x, y); // Invoke m with arguments x and y

    System.out.println("x is " + x);
    System.out.println("y[0] is " + y[0]);
  }

  public static void m(int number, int[] numbers) {
    number = 1001; // Assign a new value to number
    numbers[0] = 5555; // Assign a new value to numbers[0]
  }
}
```

x is 1
y[0] is 5555

Heap

The Java Virtual Machine stores the array in an area of memory called the *heap*.

Call Stack and Heap



- **When invoking <u>m(x, y)</u>, the values of <u>x</u> and <u>y</u> are passed to <u>number</u> and <u>numbers</u>. Since <u>y</u> contains the reference value to the array, <u>numbers</u> now contains the same reference value to the same array.**
- **The JVM stores the array in an area of memory, called *heap*, which is used for dynamic memory allocation where blocks of memory are allocated and freed in an arbitrary order.**

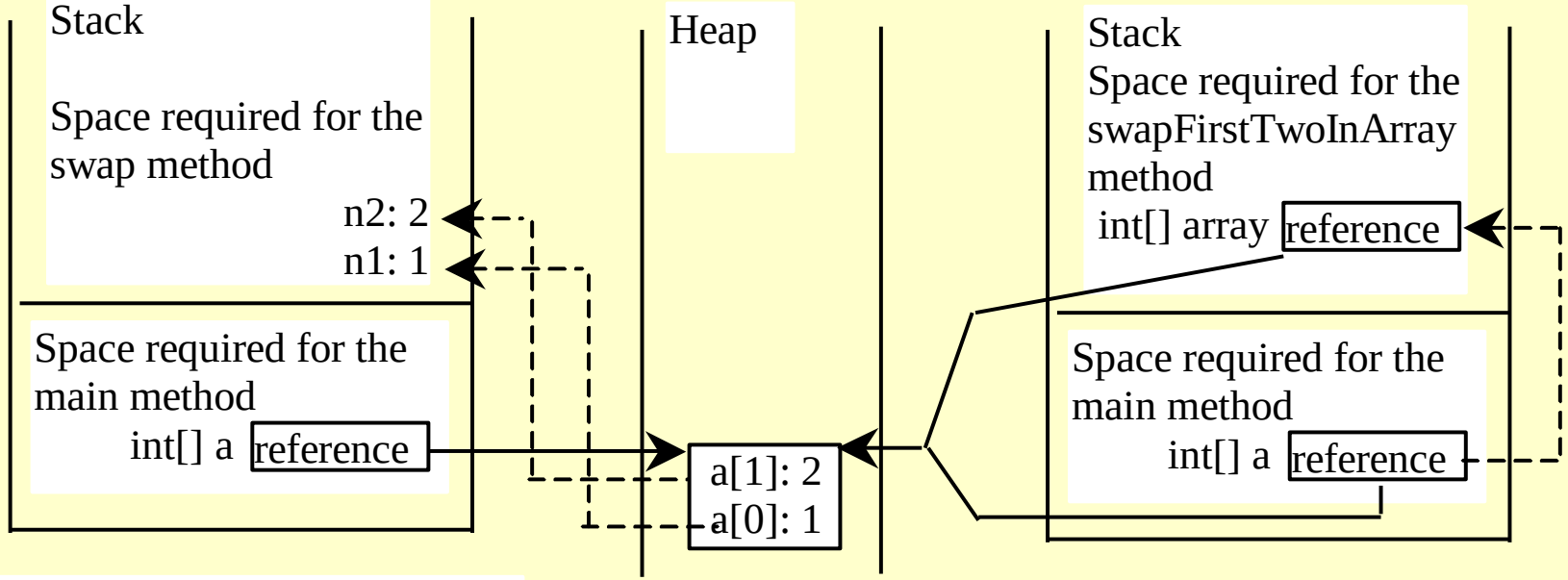
Passing Arrays as Arguments
Lab 5 – Exercise 1 – Question 1(ii)

```
public class Lab5a {
  public static void main(String[] args) {

    int[] a = {1,2}; // a represents an array of int values

    System.out.println("Before invoking swap");
    System.out.println("aray is {" + a[0] + ", " + a[1] + "}");
    swap(a[0],a[1]);
    System.out.println("After invoking swap");
    System.out.println("aray is {" + a[0] + ", " + a[1] + "}");

    System.out.println("Before invoking swapFirstTwoInArray");
   System.out.println("aray is {" + a[0] + ", " + a[1] + "}");
   swapFirstTwoInArray(a);
   System.out.println("After invoking swap");
    System.out.println("aray is {" + a[0] + ", " + a[1] + "}");
  }
```

Passing Arrays as Arguments
Lab 5 – Exercise 1 – Question 1(ii)

```
public static void swap(int n1, int n2) {
    int temp = n1;
    n1 = n2;
    n2 = temp;
  }

  public static void
  swapFirstTwoInArray(int[] array) {
      int temp = array[0];
      array[0] = array[1];
      array[1] = temp;
  }
}
```

Example, cont.



Invoke swap(int n1, int n2). The primitive type values in a[0] and a[1] are passed to the swap method.

The arrays are stored in a heap.

Invoke swapFirstTwoInArray(int[] array). The reference value in a is passed to the swapFirstTwoInArray method.

Returning an Array Reference
Lab 5 – Exercise 1 – Question 1(iii)

- A method can return a reference to an array.
- The return type of the method must be declared as an array of the right type.

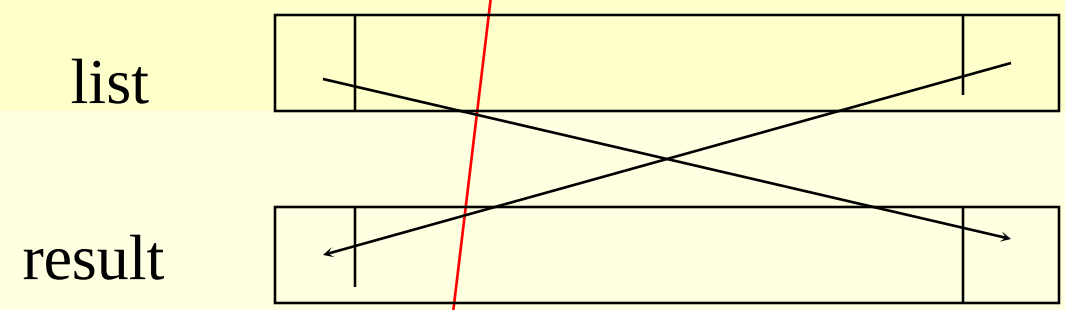
```
public static int[] reverse(int[] list) {
    int[] result = new int[list.length];
     for (int i = 0, j = result.length - 1;
         i < list.length; i++, j--) {
      result[j] = list[i];
    }
    return result;
```

- The `reverse()` method is a public static method that returns an array of int.

- **Create a new class**
- **Create a main method**
- **Create an array with size 5**
- **Print the values of the array**
- **Call method reserve and pass the array**
- **Again, print the values of the array**

Returning an Array from a Method
Lab 5 – Exercise 1 – Question 1(iii)

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];
  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }
  return result;
}
```




```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

Anonymous Array

The statement

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

creates an array using the following syntax:

```
new dataType[]{literal0, literal1, ..., literalk};
```

There is no explicit reference variable for the array. Such array is called an *anonymous array*.

Variable-Length Argument Lists

- **JDK 1.5 enables you to pass a variable number of arguments of the same type to a method.**
- **The parameter in the method is declared as follows:** `typeName ... parameterName`
- **Only one variable-length parameter may be specified in a method, and this parameter must be the last parameter.**
- **Java treats variable-length parameter as an array.**

Variable-Length Argument Lists
Lab 5 – Exercise 1 – Question 1(iv)

```
public class Lab7 {
  public static void main(String[] args) {

    printMax(21.8,32.2,13.0,45.2,25.8,36.5);
    printMax(new double[]{1,4,5,2});
  }

  public static void printMax(double ... numbers) {
    if (numbers.length == 0) {
      System.out.println("No argument passed");
      return;
    }

    double result = numbers[0];

    for (int i=1; i < numbers.length; i++)
        if (numbers[i] > result)
          result = numbers[i];

   System.out.println("The max value is " + result);
 }
}
```

Array of Objects

Lecture 14

Based on Slides of Dr. Norazah Yusof

Array of Objects

- **We can create an array of object.**
- **By combining the advantage of array, and objects, we can build program more clean and better logical organization**

Array of Objects

- **Assume we need to store collection of 50 students, for example, we need to use several different arrays (one for names, one for address, one for matrix number and so forth.) which is very cumbersome and prone to error**
- **The better way is by creating an <u>array of students object</u>. This will result a more concise and easier-to-understand code.**

Array of objects declaration

- **Declaration of array: 2 ways**
- **Way one: involve 2 steps:**

  1) Step 1: Creation of array of type class (to hold objects of that class type).

  - **Syntax:**

```
ClassName [] ArrayName = new
ClassName[ArraySize];
```

  - **Example:**

```
Employee emp[] = new Employee[5];
```

Array of objects declaration

2) <u>Step 2: Creation of object in each / particular index of the array.</u>

- **Syntax:**

```
ArrayName[index]=new ClassName();
```

- **Example:**

```
emp[0] = new Employee(101,12000);
emp[1] = new Employee(102,25000);
:
emp[4] = new Employee(105,38000);
```

Array of objects declaration and Initialization

- **Second Way: Initializer list.**
- **<u>Creation of array of type class and initialize it using the constructor of the class.</u>**
- **Syntax:**

```
ClassName [] ArrayName  ={new ClassName(1,1),
new ClassName(1,0)};
```

- **Example:**

```
Rectangle[] rect1  ={new Rectangle(4,2), new
 Rectangle(3,3), new Rectangle(6,7)};
```

Two Dimensional Array

Lecture 15

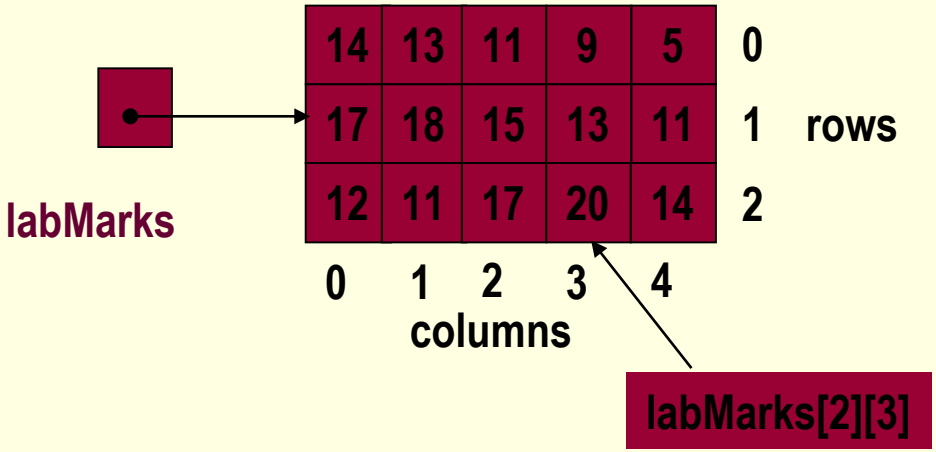
Based on Slides of Dr. Norazah Yusof

Two-dimensional Arrays

- Arrays can be extended to several dimensions (Two or more).
- A two dimensional array, for example, is a repetition of one-dimensional arrays (i.e. array of arrays).

```
int[][] labMarks = {
                {14, 13, 11, 9, 5},
                {17, 18, 15, 13, 11},
                {12, 11, 17, 20, 14}
              }
```

array of three rows and five columns



labMarks is a two dimensional array of marks of three students in five lab tests. Student numbers are represented along rows from 0 to 2. Lab tests are numbered along five columns. Each array cell contains student's marks.

Declaration of Two-dimensional Arrays

```
// Declare array ref var
dataType[][] refVar;
// Create array and assign its reference to variable
refVar = new dataType[5][5];

            or

// Combine declaration and creation in one statement
dataType[][] refVar = new dataType[5][5];
// Alternative syntax
dataType refVar[][] = new dataType[5][5];
```

Declaring Variables of Two-dimensional Arrays and Creating Two-dimensional Arrays

Example:

```
int[][] matrix = new int[5][5];
```

or

```
int matrix[][] = new int[5][5];
```

Accessing Two-Dimensional Array

```
for (int i = 0; i < matrix.length; i++)
  for (int j = 0; j < matrix[i].length; j++)
    matrix[i][j] = (int)(Math.random() * 100);
```

Two-dimensional Array Illustration

	0	1	2	3	4
0					
1					
2					
3					
4					

```
matrix = new int[5][5];
```

	0	1	2	3	4
0					
1					
2		7			
3					
4					

```
matrix[2][1] = 7;
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

```
int[][] array = {
   {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9},
   {10, 11, 12}
};
```

matrix.length? 5

matrix[0].length? 5

array.length? 4

array[0].length? 3

Declaring, Creating, and Initializing Using Shorthand Notations

You can also use an array initializer to declare, create and initialize a two-dimensional array. For example,

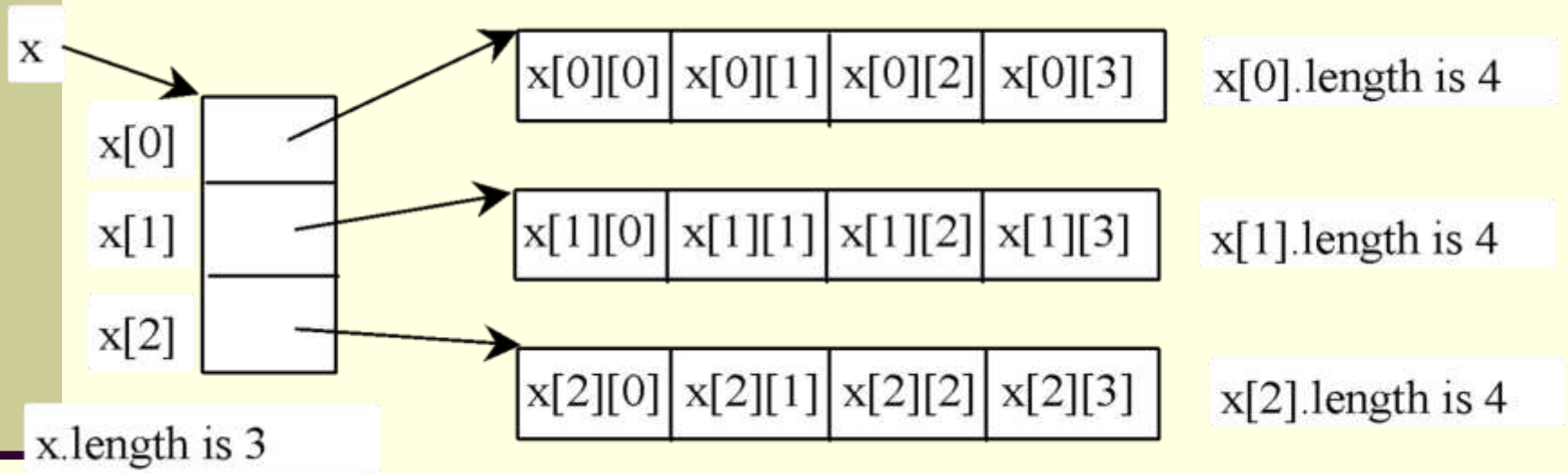
```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

Same as

```
int[][] array = new int[4][3];
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

Lengths of Two-dimensional Arrays

int[][] x = new int[3][4];



Lengths of Two-dimensional Arrays, cont.

```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

array.length

array[0].length

array[1].length

array[2].length

array[3].length

array[4].length ArrayIndexOutOfBoundsException

Processing Two-Dimensional Arrays

- Declare a two-dimensional array named matrix of type integer with 3 rows and 4 columns:

```
int[][] matrix = new int[3][4];
```

- Some processing examples:

1. Input arrays with values read from the keyboard.
2. Printing arrays.
3. Summing all elements.
4. Summing elements by column
5. Finding which row has the largest sum.

Accessing Two-Dimensional Array Elements

- **Programs that process two-dimensional arrays can do so with nested loops.**
- **To fill the matrix array:**

```
for (int row = 0; row < matrix.length; row++)
{

  for (int col = 0; col < matrix[row].length; col++)
  {
    System.out.print("Enter a value: ");
    matrix[row][col] = keyboard.nextInt();
  }
}
```

Number of rows, not the largest subscript

Number of columns, not the largest subscript

`keyboard` references a `Scanner` object

Accessing Two-Dimensional Array Elements

- **To print out the `matrix` array:**

```
for (int row = 0; row < matrix.length;
  row++)
{
   for (int col = 0; col <
  matrix[row].length; col++)
  {
     System.out.println(matrix[row][col]);
   }
}
```

Summing The Elements of a Two-Dimensional Array

```
int[][] numbers = { { 1, 2, 3, 4 },
                    {5, 6, 7, 8},
                    {9, 10, 11, 12} };
int total;
total = 0;
for (int row = 0; row < numbers.length; row++)
{
  for (int col = 0; col < numbers[row].length;
  col++)
    total += numbers[row][col];
}
System.out.println("The total is " + total);
```

Ragged Arrays

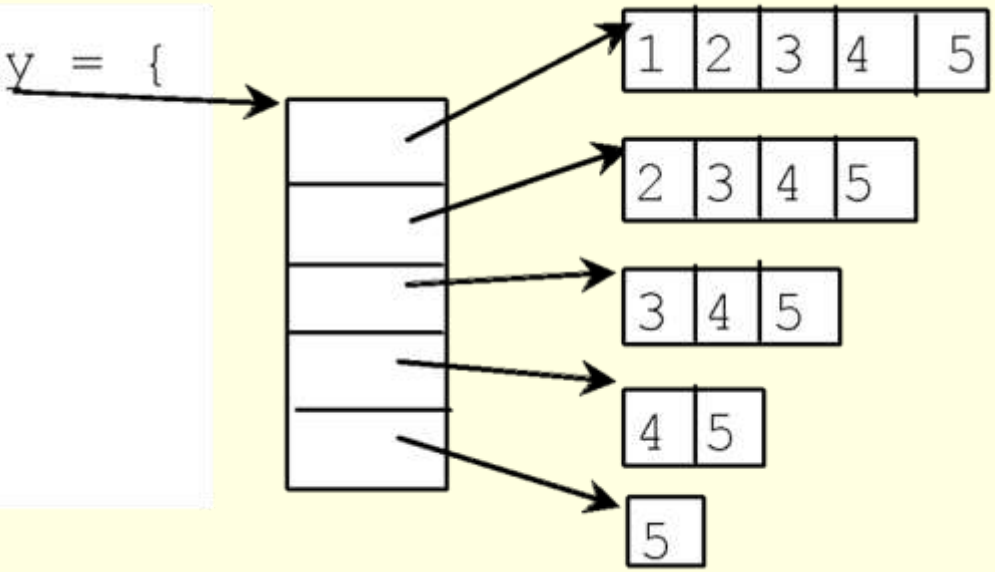
Each row in a two-dimensional array is itself an array. So, the rows can have different lengths. Such an array is known as *a ragged array*. For example,

```
int[][] matrix = {
  {1, 2, 3, 4, 5},
  {2, 3, 4, 5},
  {3, 4, 5},
  {4, 5},
  {5}
};
```

```
matrix.length is 5
matrix[0].length is 5
matrix[1].length is 4
matrix[2].length is 3
matrix[3].length is 2
matrix[4].length is 1
```

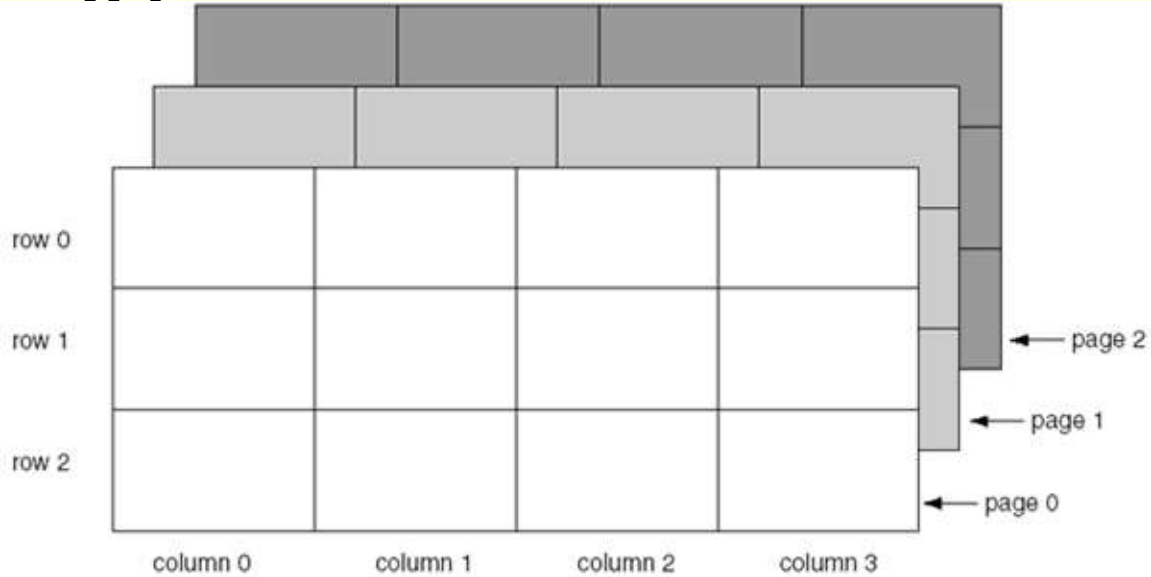
Ragged Arrays, cont.

```
int[][] triangleArray = {
   {1, 2, 3, 4, 5},
   {2, 3, 4, 5},
   {3, 4, 5},
   {4, 5},
   {5}
};
```



More Than Two Dimensions

- **Java does not limit the number of dimensions that an array may be.**
- **More than three dimensions is hard to visualize, but can be useful in some programming problems.**



Using the `Arrays` Class

- **`Arrays` class**
  - **Contains many useful methods for manipulating arrays**
  - **`static` methods**
    - **Use with class name**
    - **Without instantiating `Arrays` object**
  - **`binarySearch()` method**
    - **Convenient way to search through sorted lists of values of various data types**
    - **List must be in order**

Useful Methods of the `Arrays` Class

Method	Purpose
`static int binarySearch(type [] a, type key)`	Searches the specified array for the specified key value using the binary search algorithm
`static boolean equals(type[] a, type[] a2)`	Returns `true` if the two specified arrays of the same type are equal to one another
`static void fill (type[] a, type val)`	Assigns the specified value to each element of the specified array
`static void sort (type[] a)`	Sorts the specified array into ascending numerical order
`static void sort(type[] a, int fromIndex, int toIndex)`	Sorts the specified range of the specified array into ascending numerical order

Table 8-3 Useful methods of the `Arrays` class

Exercises

- **Write two java program to create and print this array:**

  1 2 3 4 5

  6 7 8 9 10

  11 12 13 14 15

  16 17 18 19 20

1. **Using one statement to declare, create and initialize the array and nested for loop to print the output**
2. **Using Scanner class to get the input values from the user to initialize the array and nested for loop to print the output**

```
public class Array {
        public static void main(String[] args)   {
            int[][] a = {{1, 2, 3, 4, 5},
                         {6, 7, 8, 9, 10},
                         {11, 12, 13, 14, 15},
                         {16, 17, 18, 19, 20}};

                        for(int i=0; i<a.length; i++) {
                              for(int j=0; j<a.length;j++) {
                                 System.out.print(a[i][j]+" ");
                                    }
        System.out.println("");
          }
   }
}
```

ArrayList

Lecture 17

Based on Slides of Dr. Norazah Yusof

The `ArrayList` Class

- Similar to an array, an `ArrayList` allows object storage
- Unlike an array, an `ArrayList` object:
  - Automatically expands when a new item is added
  - Automatically shrinks when items are removed

- Requires:

```
import java.util.ArrayList;
```

Creating and Using an `ArrayList` and adding items using `add()` method

- Create an `ArrayList` object with no-args constructor
  - `ArrayList townList = new ArrayList();`
- To add element to the `ArrayList`, use the `add` method:
  - `townList.add("Kangar");`
  - `townList.add("Alor Setar");`
- To get the current size, call the `size` method
  - `townList.size();  // returns 2`
- Example: Lab 6 – Exercise 1 – Question 3

Accessing & Removing items in an `ArrayList`

- To access items in an `ArrayList`, use the `get` method as follows:

```
townList.get(1); // 1 is the index of the item to get.
```

- A loop is used in the following statement to access every element in the `ArrayList` named `townList`.

```
for(int i=0;i < townList.size();i++)
        System.out.print(townList.get(i)+" ");
```

- To remove items in an `ArrayList`, use the `remove` method

```
townList.remove(1); //This statement removes the second item.
townList.remove("Penang"); //This statement removes the item
                           // with the value "Penang".
```

Adding and replacing existing items using two argument method

- The `ArrayList` class's `add` method with one argument adds new items to the end of the `ArrayList`

- To insert items at a location of choice, use the `add` method with two arguments:

```
townList.add(6, "Shah Alam");
```

  This statement inserts the `String` "Shah Alam" at index 6

- To replace an existing item, use the `set` method:

```
townList.set(1, "Muar");
```

  This statement replaces the value at index 1 with “Muar”

Using `toString()` method

- The `ArrayList` class's `toString` method returns a string representing all items in the `ArrayList`

```
System.out.println(townList);
```

This statement yields :

```
[Muar, Alor Setar]
```

Using a Cast Operator with the `get` Method

- An `ArrayList` object is not typed
- To retrieve items from an `ArrayList`, you must cast the item to the appropriate type

```
ArrayList nameList = new ArrayList();
townList.add("Kluang"); // Inserts an item
String str = (String)townList.get(0);
```

- Try `get` without the cast to see the effect.

Using `ArrayList` as a Generic Data Type

- You can create a type-safe `ArrayList` object by using generics.
- For example an `ArrayList` object for `String`:

```
ArrayList<String> nameList = new
  ArrayList<String>();
```

- The `get` method no longer requires casts to work.

ArrayList Methods

- add(Object o) – add new object into ArrayList
- get(int index) - retrieves object reference from ArrayList *index* position
- size() - returns ArrayList size
- remove(int index) - removes the element at the specified position in this list. Shifts any subsequent elements to the left and returns the element that was removed from the list.
- indexOf(Object o) - finds the index in this list of the first occurrence of the specified element
- clear() - removes all of the elements

Exercises

- Do Exercise 1:
  - Question 1, page 115
  - Question 2, page 116
  - Question 3, page 117

Question 1, page 115

```
import java.util.ArrayList;

public class CircleArrayList {
    private double radius;

    public CircleArrayList(double radius)  {
        this.radius=radius;
    }

    public void setRadius() {
         this.radius=radius;
    }

    public double getRadius() {
         return radius;
    }

    public double getArea()  {
         return (Math.PI*radius*radius);
    }
}
```

Question 1, page 115

```
import java.util.ArrayList;

public class TestCircleAL {
  public static void main(String[] arg)       {
    ArrayList cList = new ArrayList();

    cList.add(new CircleArrayList(2.3));
    cList.add(new CircleArrayList(3.3));

    System.out.print("The area : %2.2f " +
                     ((CircleArrayList)cList.get(0)).getArea());
  }
}
```

Question 2, page 116

```
import java.util.ArrayList;
ArrayList student = new ArrayList();

student.add("Siti Rahimah");
student.add("Robert Lau");

System.out.print(student);
Student.set(1, "Muhammad");
```